Importance of Security Analysis

We acknowledge funding provided by Microelectronic Commons and the support of OUSW(R&E).

What can go wrong in a securely designed IoT system?

We designed an IoT system securely with a strong threat model.

What can go wrong?


To err is
human or LLM

We can make mistakes!

- Coding errors.
  - `if (uid = 0) { ...provide root access…}`

- Incorrect or Misconfigurations.
  - Not updating CRL.

Misconfigurations drive 80% of security exposures

- Insecure dependencies.
  - Using library with existing bugs.

Defects! Bugs! Vulnerabilities!





Defects! Bugs! Vulnerabilities!

- **Defect**: System not doing what is expected.
  - Not turning on AC when temperature increases.

- **Bug**: System doing something incorrectly.
  - Increasing temperature by 2 times the set value.

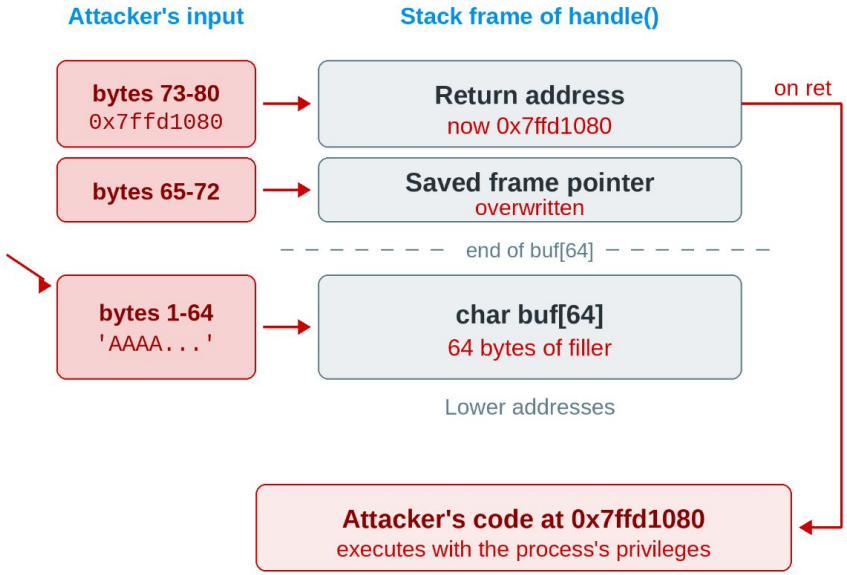
- **Vulnerability**: Bug that can be exploited to gain control of the system.
  - Using temperature value as an index into an array.

Memory corruption vulnerability.

```
void handle(char *input) {
    char buf[64];

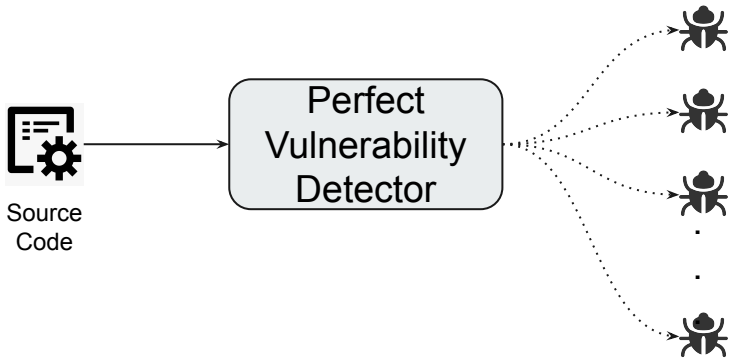
    strcpy(buf, input);
    /* no bounds check */

    return;
}
```



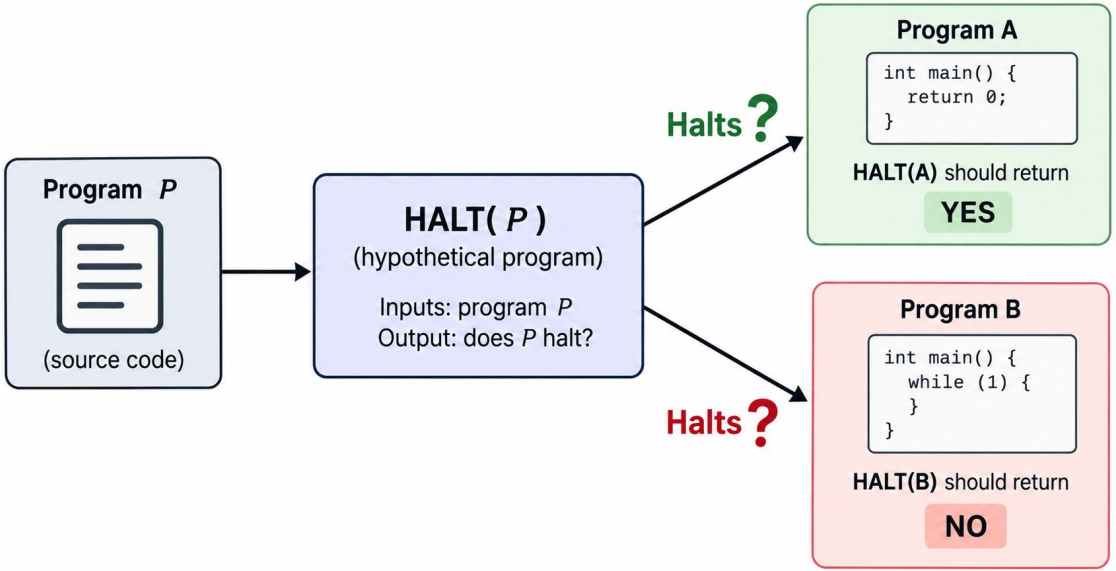
The input is longer than buf[64], so its last 8 bytes land on the saved return address.
When handle() returns, the CPU jumps to whatever address the attacker put there.

Can we automatically find all these vulnerabilities?



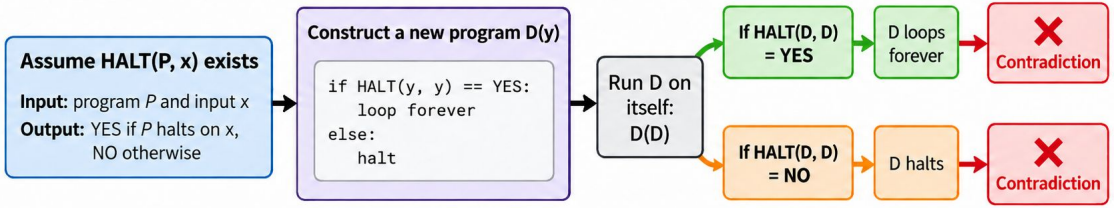
Perfect Vulnerability Detector!

Halting problem!



Perfect Vulnerability Detector!

Impossibility of Halting problem!



Perfect Vulnerability Detector!

Is there an array out of bounds access in the below program? Yes/No

```
int arr[10];
if (HALT(I, I) == YES) {
    arr[20] = 1;
} else {
    arr[2] = 1;
}
```

Perfect Vulnerability Detector!

Is there an array out of bounds access in the below program? Yes/No; Undecidable!

```
int arr[10];
if (HALT(I, I) == YES) {
    arr[20] = 1;
} else {
    arr[2] = 1;
}
```

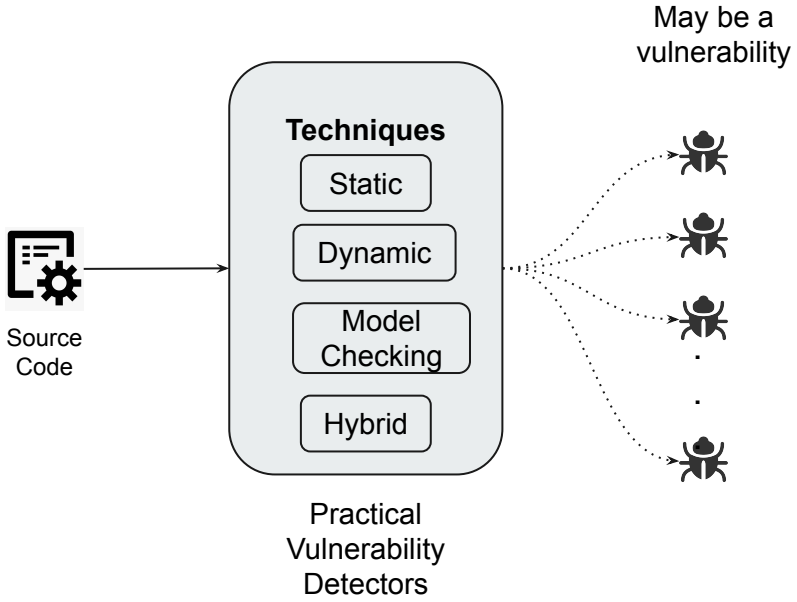
Practical Vulnerability Detector

How do we find bugs/vulnerabilities?

Practical Vulnerability Detector

- Looking at the code and analyzing it => **Static analysis**.
- Testing (run the program and check output) => **Dynamic analysis**.
- Write a model for the program and check the model => **Model checking**.
- Combining the above => **hybrid**.

Practical Vulnerability Detector



Design choices for vulnerability detection

Impossible

True Result	Sound	Complete	Neither sound nor complete	Sound and Complete
Bug	Bug	May or May not be a bug.	May or May not be a bug.	Bug
Not a bug	May or May not be a bug.	Not a bug.	May or May not be a bug.	Not a bug
	false positives No false negatives	No false positives false negatives	false positives false negatives	No false positives No false negatives

Precision and Recall

		Analysis Outcome	
		Accept	Reject
Program's Ground Truth	Good	True Negative	False Positive
	Bad	False Negative	True Positive

$$precision = \frac{\#\ \text{True Positives}}{\#\ \text{Rejected}}$$

$$recall = \frac{\#\ \text{True Positives}}{\#\ \text{Bad}}$$

Design choices for vulnerability detection

		Recall=1	Precision=1		
True Result	**Sound**	**Complete**	**Neither sound nor complete**	**Sound and Complete**
Bug	Bug	May or May not be a bug.	May or May not be a bug.	Bug
Not a bug	May or May not be a bug.	Not a bug.	May or May not be a bug.	Not a bug
	false positives No false negatives	No false positives false negatives	false positives false negatives	No false positives No false negatives

Static Analysis

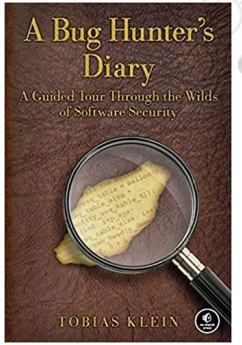
- Static w.r.t to the software being analysed:
  - We do not run (or dynamically execute) the program.

- Example:
  - grep -r "sscanf[^)]*,[^)]*%s"
  - To find: CWE-120 - Buffer Copy without Checking Size of Input ('Classic Buffer Overflow').

Static Analysis

- `grep -r "sscanf[^)]*,[^)]*%s"`

```
static char cs;
...
int ret = sscanf(buf, "%s", &cs);
if (ret != 1) {
   accdet_error("..");
   return -1;
}
...
```



Most successful technique

CVE-2016-8472: In MediaTek Kernel Driver

Comparing Static Analysis

```
void foo(unsigned i) {
  int a[2];
 if (i < 2) a[i] = 0; //p3
 else a[i] = 1; //p4
}

int main() {
  unsigned i, j;
  scanf("%u %u", &i, &j);
  if (i < 2) foo(i); //p1
  foo(j); //p2
  return 0;
}
```

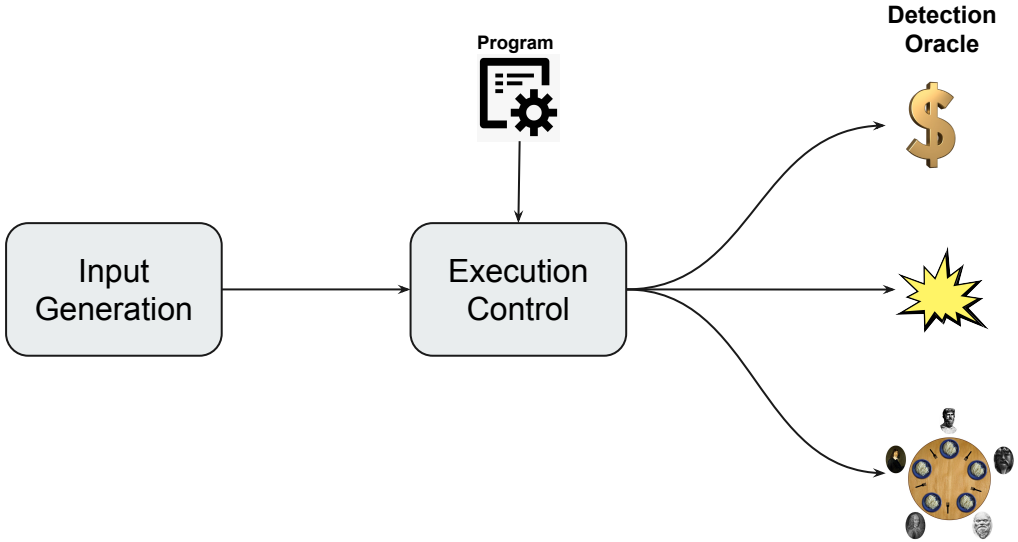
Consider the following out-of-bounds detectors with the following warnings at corresponding lines:

- SA1: P1, P2, P3, P4
- SA2: P3 and P4
- SA3: P4
- SA4: P4 only when called from P2

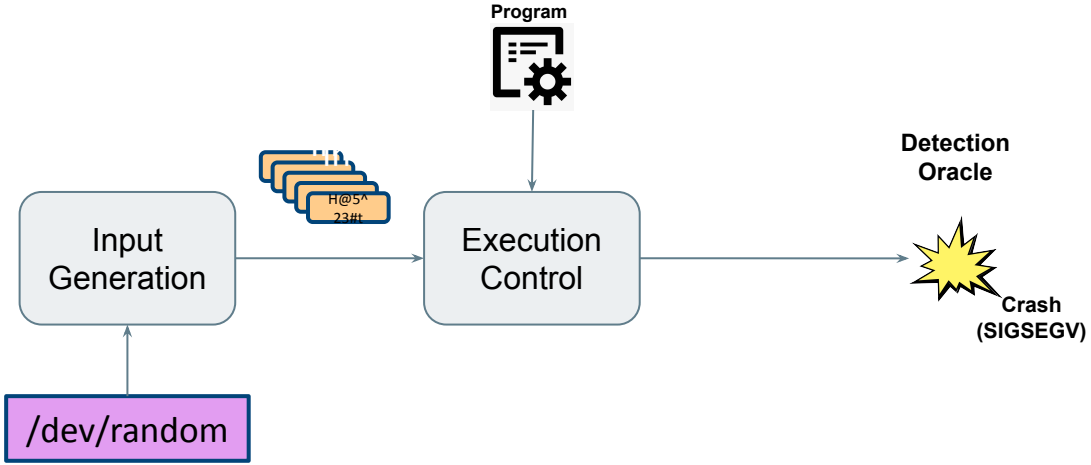
Are these analyses sound?

What about precision?

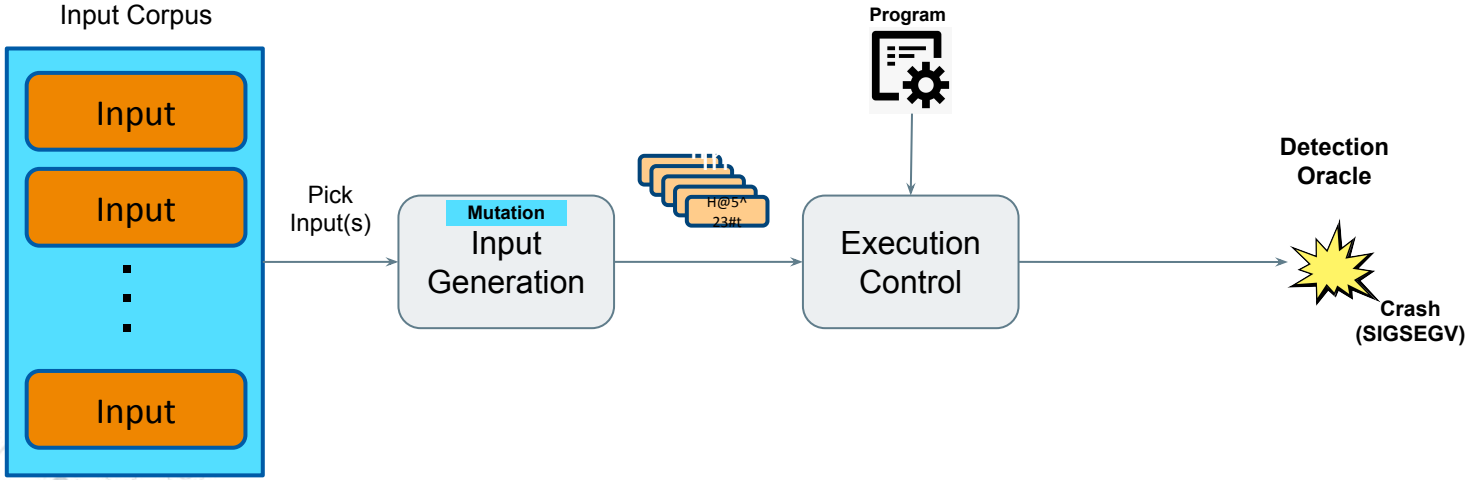
Dynamic Analysis or Automated Testing (Fuzzing)



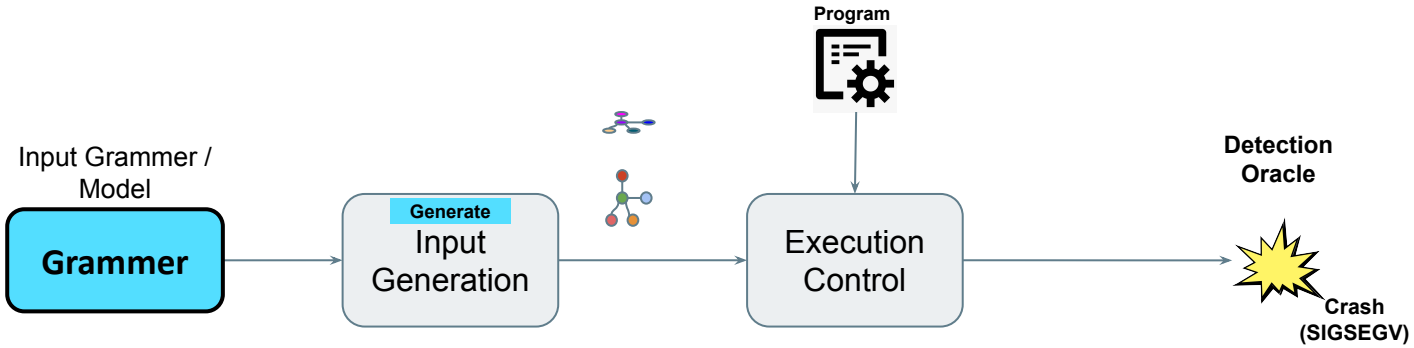
Simple Random Testing Fuzzing: Random data



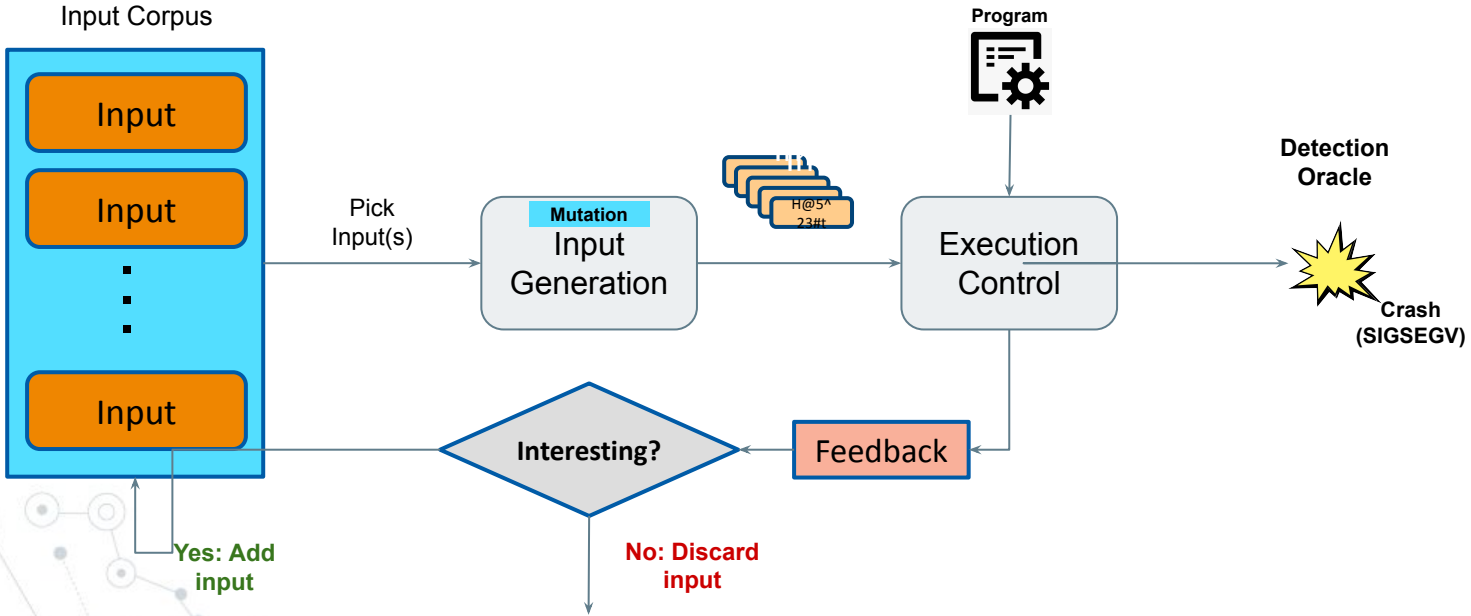
Mutation based fuzzing



Generation based fuzzing

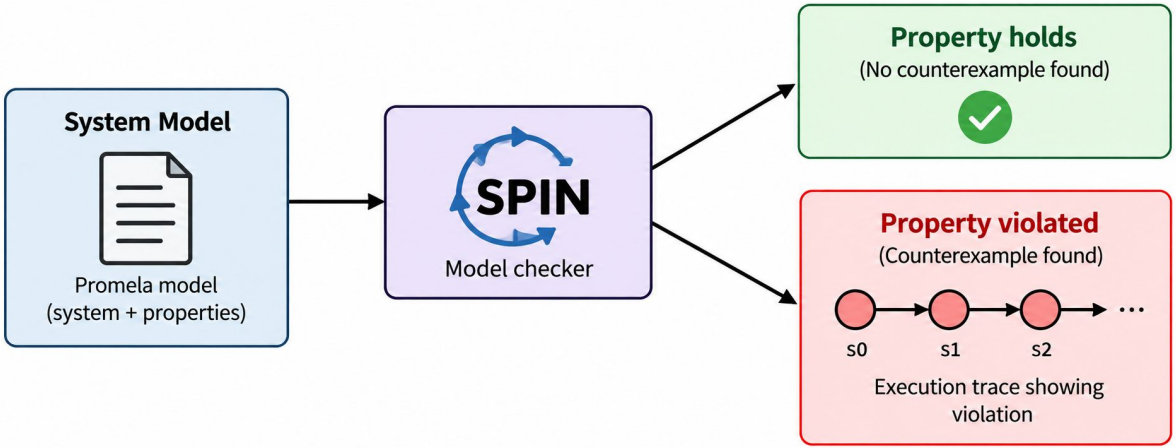


Feedback guided Mutation based fuzzing



Model Checking

Convert program into a model and check properties on the model.



Spin

C code

Promela model

spin -a model.pml

gcc -O2 -o pan pan.c

./pan

CBMC - C Based Bounded Model Checking

Code Example

```
cbmc example.c --bounds-check --trace
```

CBMC in Real World

Free RTOS:
https://github.com/FreeRTOS/FreeRTOS/tree/main/FreeRTOS/Test/CBMC/proofs

FreeRTOS+TCP:

https://github.com/FreeRTOS/FreeRTOS-Plus-TCP/tree/main/test/cbmc/proofs

Why don’t we use model checking?